

Suppose we want a gradient that shades from top to bottom. When you create the gradient object, keep the x values (1st and 3rd parameters) constant, and make the y values (2nd and 4th parameters) range from 0 to the height of the canvas.

```
• var my_gradient = context.createLinearGradient(0, 0, 0, 225);
• my_gradient.addColorStop(0, "black");
• my_gradient.addColorStop(1, "white");
• context.fillStyle = my_gradient;
• context.fillRect(0, 0, 300, 225);
```

Pattern

The HTML5 Canvas API also includes an option to set an image as a repeatable pattern for either a path stroke or fill. To create a pattern with the HTML5 Canvas, we can define a pattern with the `createPattern()`, set it to the fill style using the `fillStyle` property, and then fill a shape using `fill()`. The `createPattern()` method requires an image object and a repeat option, which can be `repeat`, `repeat-x`, `repeat-y`, or `no-repeat`.

What you don't see

Pixel Data

One of the most useful—albeit nonobvious—aspects of the Canvas API is the ability for developers to easily access the underlying pixels in the canvas. This access is both a good and a bad thing: it is easy to get access to the pixel value, and it is equally easy to modify those values and apply them back to the canvas. In fact, you can manipulate the canvas entirely through the pixel value calls and forget all about the different rendering calls we've discussed in this chapter. This is made possible by the existence of three functions on the context API –

- ▶ `context.getImageData(sx, sy, sw, sh)`
- ▶ `context.putImageData(imagedata, dx, dy)`
- ▶ `context.createImageData(sw, sh)`

Canvas Security

Although most developers would use pixel manipulation for legitimate means, it is quite possible that the ability to fetch and update data from a canvas could be used for not-so-legitimate purposes as well. For this reason, the concept of an origin-clean canvas was specified. What it does is that for any canvas tainted with images from origins other than the source of the